

INFORME TÉCNICO DE AVANCE · FASE 3 DISEÑO ALGORÍTMICO, COMPLEJIDAD Y ARQUITECTURA TÉCNICA

Proyecto: Duración de la titulación en el pregrado universitario chileno (2025)
Evaluación Sumativa 2 · Semana 2

Integrantes (Grupo 6):	Fernanda Ovalle Román Sebastián Cajales Cid César Lorca Bacián Jorge Álvarez Ossandón
Docente responsable:	Dr. Omar Salinas Silva
Programa académico:	Magíster en Ciencia de Datos e Inteligencia Artificial
Asignatura:	MCDI500 – Programación para la Ciencia de Datos
Institución:	Universidad Andrés Bello · UNAB Online
Fecha de entrega:	25 de septiembre de 2026
Repositorio GitHub:	https://github.com/Feroroman/proyecto-grupo6-mcdi500

ÍNDICE GENERAL

I. Introducción y contextualización del avance	3
II. Conjunto de datos y estructuras para consumo algorítmico	3
III. Arquitectura modular y diseño orientado a objetos del pipeline	4
3.1 Principios de diseño modular en el paquete src/	4
3.2 Jerarquía de clases del pipeline (Transformador y Pipeline)	4
3.3 Diagrama de dependencias técnicas y justificación de POO	5
IV. Diseño e implementación de algoritmos estructurados y recursivos	5
4.1 Enfoque Divide and Conquer: Merge Sort y Quickselect	5
4.2 Exploración jerárquica de combinaciones críticas con poda	5
4.3 Agregación anidada: jornada dentro de área de conocimiento	6
V. Medición empírica de complejidad computacional y eficiencia	6
5.1 Complejidad temporal y ajuste frente a cotas asintóticas	6
5.2 Complejidad espacial y profundidad de la pila de llamadas (Call Stack)	6
5.3 Bucle iterativo vs. vectorización en agrupación y One-Hot	7
VI. Validación técnica, verificación de resultados y pruebas unitarias	7
6.1 Cobertura de casos normales, límite y excepciones	7
6.2 Verificación numérica y análisis de sensibilidad de casos imputados	8
VII. Trabajo colaborativo, trazabilidad y control de versiones	8
VIII. Conclusiones y proyección hacia la Fase 4	9
IX. Bibliografía (Norma APA 7.^a)	9
Anexos técnicos	10
Anexo A · Trazabilidad y verificación automatizada del repositorio	10
Anexo B · Registro de pruebas automatizadas y cobertura	11

I. Introducción y contextualización del avance

El presente informe técnico da cuenta de la Evaluación Sumativa 2, correspondiente a la Fase 3 del proyecto transversal del curso MCDI500 (Programación para la Ciencia de Datos), titulado «Duración de la titulación en el pregrado universitario chileno (2025)». En las Fases 1 y 2, el equipo formalizó la pregunta analizable del proyecto, accedió al registro administrativo oficial de datos abiertos del Ministerio de Educación de Chile (328.998 filas × 40 columnas) y construyó un pipeline de preprocesamiento, limpieza, imputación justificada y escalamiento robusto, obteniendo un conjunto de datos depurado de 105.060 registros de titulados universitarios de pregrado y 84 variables analíticas.

La Fase 3 representa un salto cualitativo fundamental: el paso desde la manipulación tabular hacia el diseño algorítmico estructurado, el análisis asintótico de complejidad y la ingeniería de software orientada a objetos (POO). Conforme a las directrices de la asignatura, el trabajo de esta etapa aborda directamente la necesidad de calcular estadísticos de localización (cuantiles y medianas) de manera eficiente sobre grandes volúmenes de datos, explorar combinaciones de factores académicos de alto riesgo sin incurrir en una explosión combinatoria, instrumentar empíricamente el consumo de tiempo y memoria, y consolidar el pipeline de datos en una jerarquía de clases robusta.

Asimismo, este avance integra de forma explícita las discusiones del Foro Técnico de la Semana 1, analizando el compromiso (trade-off) entre implementaciones iterativas y recursivas, la sobrecarga espacial de la pila de llamadas (call stack), el riesgo de desbordamiento de recursión frente a los límites del intérprete Python (sys.getrecursionlimit) y las ventajas comparativas de la vectorización en NumPy/pandas frente a la iteración imperativa convencional.

II. Conjunto de datos y estructuras para consumo algorítmico

El sustrato empírico sobre el cual operan los algoritmos de la Fase 3 es el dataset oficial depurado en la Fase 2 (titulados_2025_pregrado_univ_limpio.csv), alojado localmente en data/processed/. Este subconjunto contiene 105.060 titulados reales con 84 columnas resultantes de las transformaciones de ingeniería de variables (codificación One-Hot de nominales, codificación ordinal respetando jerarquías y escalamiento robusto mediante mediana y RIC).

Para garantizar la eficiencia algorítmica y evitar transferencias de memoria innecesarias, los algoritmos implementados consumen únicamente las estructuras de datos estrictamente requeridas para sus operaciones:

Estructura / Variable	Tipo de Dato	Rol en los Algoritmos de Fase 3
dur_total_carr	float / Sequence[float]	Variable de respuesta continua (semestres reales hasta titulación). Consumida por Merge Sort y Quickselect.
nomb_area_cine_amb	category / string	Factor académico jerárquico de nivel superior (10 grandes áreas del conocimiento UNESCO).

nomb_jornada, nomb_modalidad	category / string	Factores de régimen curricular analizados en la exploración de sobreduración y agregación anidada.
anio_ing_carr_ori_imputada	bool (0/1)	Bandera de trazabilidad que aísla los 13.396 casos originalmente con código 1900 para el análisis de sensibilidad.

[INTEGRIDAD DE DATOS]

De acuerdo con las reglas de reproducibilidad institucional, ningún archivo CSV se encuentra versionado en el repositorio Git (por superar el límite y constituir artefactos generables). El notebook F1 genera el subconjunto crudo y F2 regenera la matriz limpia de 105.060×84 , garantizando que todo el pipeline algorítmico se ejecute de inicio a fin desde cero.

III. Arquitectura modular y diseño orientado a objetos del pipeline

3.1 Principios de diseño modular en el paquete src/

La arquitectura del proyecto sigue el principio de responsabilidad única (Single Responsibility Principle) y modularidad desacoplada. Toda la lógica de negocio, manipulación de datos, algoritmos e instrumentación reside en scripts reutilizables dentro del paquete src/, dejando los notebooks de Jupyter exclusivamente como capas de orquestación, experimentación interactiva y narrativa técnica.

Módulo	Responsabilidad Principal	Lógica Clave / Clases
src/algoritmos.py	Ingeniería algorítmica estructurada y recursiva.	merge_sort, cuantil_quickselect, explorar_con_poda, agregacion_anidada.
src/complejidad.py	Perfilado experimental y benchmarking empírico.	Clase MedidorComplejidad, perfiles con time.perf_counter y tracemalloc.
src/pipeline.py	Arquitectura POO del pipeline de preprocesamiento.	Clase abstracta Transformador (fit/apply) y clase orquestadora Pipeline.
src/limpieza.py, escalado.py	Operaciones elementales de transformación tabular.	Funciones funcionales puras de imputación, codificación y escalado.
scripts/evidencias.py	Auditoría, trazabilidad y control formal automático.	Inspección AST de dependencias, auditoría de commits y ejecución de suites.

3.2 Jerarquía de clases del pipeline (Transformador y Pipeline)

En cumplimiento de las observaciones del docente sobre la Sumativa 1, la Fase 3 transforma el pipeline secuencial de funciones en una arquitectura de programación orientada a objetos (POO). Se implementa una clase base abstracta Transformador que define formalmente la interfaz con métodos fit y apply. Esta separación previene formalmente el data leakage. Cada etapa se modela como subclase (LimpiadorValoresEspeciales, ImputadorMediana, CodificadorCategorico, EscaladorRobusto), compuestas dentro de la clase orquestadora Pipeline, la cual instrumenta tiempos y consumos de memoria por etapa.

3.3 Diagrama de dependencias técnicas y justificación de POO

La independencia modular de src/ fue validada por el script de trazabilidad mediante análisis sintáctico abstracto (AST). El grafo confirma que los módulos de algoritmos y complejidad no presentan acoplamiento circular, preparando al equipo para la Fase 4 sin requerir refactorizaciones estructurales.

IV. Diseño e implementación de algoritmos estructurados y recursivos

4.1 Enfoque Divide and Conquer: Merge Sort y Quickselect

Para responder a la pregunta de investigación sobre la sobreduración de las carreras, el cálculo de medianas y percentiles es una operación central. En src/algoritmos.py se implementaron dos algoritmos recursivos basados en Divide and Conquer:

1. Merge Sort (Ordenamiento por Mezcla): Divide la lista de duraciones en dos mitades simétricas de forma recursiva, ordena cada mitad y luego las fusiona en un paso lineal. Su ecuación de recurrencia formal es:

$$T(n) = 2 \cdot T(n/2) + O(n)$$

Aplicando el Caso 2 del Teorema Maestro (donde $a = 2$, $b = 2$ y $f(n) = O(n)$), se deduce formalmente una complejidad temporal determinista de $\Theta(n \log n)$. Su costo espacial auxiliar es $O(n)$ debido a los arreglos intermedios de mezcla.

2. Quickselect (Búsqueda de Cuantiles): Algoritmo de selección probabilístico que particiona el arreglo respecto a un pivote aleatorio (método Hoare). A diferencia del ordenamiento completo, Quickselect solo se llama recursivamente sobre el subarreglo que contiene la posición k buscada. Su recurrencia promedio es:

$$T(n) = T(n/2) + O(n)$$

Por el Teorema Maestro (Caso 3), su orden de complejidad temporal esperado es lineal $O(n)$. Esto representa una optimización crítica frente a la versión ingenua (cuantil_ordenando), que ordena la totalidad de los 105.060 registros con costo $O(n \log n)$ simplemente para inspeccionar un valor.

4.2 Exploración jerárquica de combinaciones críticas con poda

El análisis multivariado de factores (área de conocimiento, jornada, modalidad y tramo etario) enfrenta un desafío combinatorio clásico: el árbol de subgrupos posibles crece de forma exponencial según el producto cartesiano de las cardinalidades. Para detectar combinaciones críticas de sobreduración sin colapsar el sistema, se diseñó la función explorar_con_poda, que recorre el árbol en profundidad implementando dos criterios de poda matemáticamente rigurosos y demostrablemente admisibles:

- Poda 1 (Por Soporte Mínimo): Si un nodo intermedio agrupa menos de un umbral representativo ($n < 100$), se poda de inmediato la rama completa. Demostración de corrección: Los descendientes de un subconjunto $S' \subseteq S$ satisfacen estrictamente $|S'| \leq |S|$. Si $|S| < 100$, ningún subgrupo derivado podrá satisfacer jamás el soporte mínimo representativo.
- Poda 2 (Por Cota Superior): Si el valor máximo de la duración dentro del subconjunto actual es menor al umbral crítico analizado ($\max(\text{dur_total}) < \text{umbral}$), la rama se poda inmediatamente. Demostración de corrección: Para cualquier subconjunto no vacío, la mediana

satisface $\text{median}(S') \leq \max(S') \leq \max(S)$. Si $\max(S) < \text{umbral}$, es matemáticamente imposible que cualquier descendiente posea una mediana superior o igual a dicho umbral.

Para validar la exactitud de la poda, se construyó la función simétrica `explorar_exhaustivo` (búsqueda ciega total). Las pruebas automatizadas demostraron que ambas funciones devuelven exactamente los mismos hallazgos críticos de sobreduración, pero la versión con poda reduce el número de nodos visitados y el tiempo de cómputo en más de un 75%.

4.3 Agregación anidada: jornada dentro de área de conocimiento

En respuesta directa a la observación formulada por el Dr. Omar Salinas Silva, se implementó `agregacion_anidada` en `src/algoritmos.py`. Esta función analiza la matriz completa de 105.060 registros para calcular la duración mediana de cada régimen de jornada curricular anidado dentro de su correspondiente gran área de conocimiento CINE-UNESCO, calculando la desviación respecto a la mediana del área (`dif_vs_area`). Los resultados confirman que en carreras de Ciencias Sociales, Educación e Ingeniería, las jornadas vespertinas exhiben una sobreduración sistemática de entre 2 y 4 semestres por sobre la mediana de la jornada diurna tradicional.

V. Medición empírica de complejidad computacional y eficiencia

5.1 Complejidad temporal y ajuste frente a cotas asintóticas

A través de la clase `MedidorComplejidad` (`src/complejidad.py`), se realizaron pruebas de estrés experimental variando sistemáticamente el tamaño muestral $N \in [500, 1.000, 2.500, 5.000, 10.000, 20.000]$ con datos reales de duración. Se registraron los tiempos de CPU en milisegundos utilizando `time.perf_counter()` de alta resolución.

Tamaño N	Quickselect (s)	Merge Sort (s)	Cuantil Ordenando (s)
500	0.0004	0.0018	0.0009
1.000	0.0008	0.0041	0.0021
2.500	0.0019	0.0112	0.0058
5.000	0.0039	0.0245	0.0129
10.000	0.0078	0.0531	0.0275
20.000	0.0152	0.1140	0.0592

El ajuste de curvas confirma empíricamente la teoría asintótica: Quickselect escala de forma estrictamente lineal $O(N)$, mientras que Merge Sort refleja la pendiente $O(N \log N)$. A $N = 20.000$, Quickselect es 7,5 veces más rápido que Merge Sort y 3,9 veces más rápido que el ordenamiento de biblioteca, optimizando sensiblemente las consultas repetitivas de medianas.

5.2 Complejidad espacial y profundidad de la pila de llamadas (Call Stack)

El consumo de memoria dinámica fue monitorizado mediante el módulo `tracemalloc` de la biblioteca estándar de Python, evaluando la memoria pico (peak memory allocation) durante la recursión. Para $N = 20.000$ elementos, Merge Sort demandó un pico de 1.84 MB de memoria

debido a la instanciación de listas intermedias, mientras que Quickselect operó con menos de 0.42 MB.

En relación con la discusión del Foro de la Semana 1 sobre el desbordamiento de la pila (Stack Overflow), se analizó la profundidad máxima de recursión alcanzada en los algoritmos divide and conquer frente al límite por defecto del intérprete (`sys.getrecursionlimit()` = 1.000). Dado que la partición divide el espacio en mitades logarítmicas, la profundidad máxima teórica y empírica observada para $N = 105.060$ es:

Profundidad = $\lceil \log_2(105.060) \rceil = 17$ niveles

Como $17 \ll 1.000$, la recursión opera con un margen de seguridad superior al 98% respecto al límite del sistema, descartando cualquier riesgo de fallo por desbordamiento de pila en la ejecución del proyecto.

5.3 Bucle iterativo vs. vectorización en agrupación y One-Hot

Uno de los requerimientos clave señalados por el docente consistió en contrastar numéricamente el desempeño de implementaciones vectorizadas frente a bucles for imperativos sobre las 105.060 filas reales del dataset en dos tareas críticas de preprocesamiento:

1. Agrupación de categorías raras (`agrupar_raras`): Consiste en evaluar 968 nombres de carreras con frecuencias inferiores a 100 registros para reclasificarlas como 'OTRA'. La versión con bucle iterativo fila por fila tardó 4.82 segundos en CPU. La implementación vectorizada mediante `Series.isin(frecuentes)` ejecutó la misma tarea en 0.038 segundos, logrando una aceleración de 126×.
2. Codificación One-Hot (`one_hot`): Expansión de 5 variables categóricas en 36 columnas binarias indicadoras. Un bucle iterativo que construye las columnas mediante asignación celda a celda tardó 14.15 segundos y generó alta fragmentación de memoria en DataFrame. La versión vectorizada utilizando `pd.get_dummies()` ejecutó en 0.112 segundos (aceleración de 126×), reduciendo drásticamente la latencia del pipeline.

VI. Validación técnica, verificación de resultados y pruebas unitarias

6.1 Cobertura de casos normales, límite y excepciones

Siguiendo los estándares de ingeniería de software exigidos en la rúbrica, todos los algoritmos y transformadores cuentan con pruebas unitarias automatizadas estructuradas en tests/. Se diseñaron casos de prueba para cada una de las tres condiciones estipuladas:

- Caso Normal: Verificación del ordenamiento correcto de listas aleatorias, cálculo exacto de medianas en distribuciones continuas y exploración jerárquica sobre DataFrames sintéticos con factores correlacionados.
- Caso Límite (Edge Cases): Listas de un solo elemento ($\text{len} = 1$), arreglos con elementos idénticos (varianza cero), listas con valores extremos y árboles donde ningún subgrupo satisface el soporte mínimo.
- Caso de Excepción: Validación de que las funciones emitan excepciones explícitas y controladas (`TypeError` al pasar tipos inválidos, `ValueError` al ingresar listas vacías o umbrales fuera de rango, y `KeyError` ante columnas ausentes).

6.2 Verificación numérica y análisis de sensibilidad de casos imputados

En la Fase 2 se identificaron 13.396 registros (12,75% del universo) cuyo año de ingreso a la carrera de origen registraba el código 1900 (faltante no aleatorio, asociado a convalidaciones o cambios curriculares), los cuales fueron imputados con la mediana condicional por año de ingreso a la carrera actual, creando la bandera `anio_ing_carr_ori_imputada`. En cumplimiento del compromiso adquirido en la entrega previa, se ejecutó el análisis de sensibilidad recalculando la duración mediana con y sin estos 13.396 casos:

Condición de Análisis	N Registros	Mediana Duración	RIC	Desviación Estándar
Muestra total (con imputación)	105.060	10.0 semestres	2.0 semestres	2.440 semestres
Muestra pura (sin casos imputados)	91.664	10.0 semestres	2.0 semestres	2.373 semestres

Los resultados demuestran la robustez del tratamiento aplicado: tanto la mediana (10 semestres) como el rango intercuartílico (2 semestres) se mantienen estrictamente invariantes entre ambos subconjuntos. La desviación estándar apenas varía un 2,82%, confirmando que la imputación no introduce sesgo ni artefactos numéricos en las conclusiones del proyecto.

VII. Trabajo colaborativo, trazabilidad y control de versiones

En estricta atención a las observaciones de la Evaluación Sumativa 1 (donde el docente enfatizó que «el criterio evalúa el historial de esta entrega, no la intención para la siguiente»), el equipo erradicó por completo los commits directos sobre la rama main y adoptó un esquema riguroso de ingeniería de software colaborativa:

1. Desacoplamiento Estricto por Archivos: Cada archivo del proyecto posee un único autor y responsable exclusivo (César en `src/algoritmos.py` y `tests/test_algoritmos.py`; Jorge en `src/complejidad.py` y `figuras`; Fernanda en `src/pipeline.py` y `notebook F3`; Sebastián en `docs/f3_s02_grupo6.docx`, `docs/f3_s02_grupo6.pdf`, `docs/verificacion.md` y `scripts/evidencias.py`). Esta arquitectura evitó el 100% de conflictos de mezcla en archivos binarios y JSON.
2. Trabajo en Ramas y Pull Requests (PR): Todo cambio entra a main exclusivamente a través de PRs con revisión cruzada formal (César revisa a Sebastián, Jorge revisa a César, Fernanda revisa a Jorge, Sebastián revisa a Fernanda). Cada PR incluye descripciones técnicas detalladas y comentarios constructivos de revisión.
3. Trazabilidad Automatizada y Cifra Única de Commits: Para resolver la inconsistencia observada en la entrega anterior (donde se citaban distintos números de commits), el equipo utilizó el script `scripts/evidencias.py`, que extrae la cifra oficial y unificada mediante la instrucción `git rev-list --count main`, garantizando consistencia absoluta entre el repositorio y el documento formal.

VIII. Conclusiones y proyección hacia la Fase 4

El avance correspondiente a la Fase 3 consolida con éxito los fundamentos algorítmicos, la eficiencia computacional y la madurez arquitectónica del proyecto. Las principales conclusiones de esta fase son:

1. Ventaja analítica de Divide and Conquer: La implementación de Quickselect demostró ser el método óptimo para la extracción de medianas condicionales, reduciendo la complejidad temporal de $O(n \log n)$ a $O(n)$ y manteniendo un consumo espacial despreciable frente a los ordenamientos ciegos.
2. Eficacia de la exploración con poda: La formalización de podas por soporte mínimo y cota superior demostró ser una solución matemática elegante frente al problema de la explosión combinatoria, permitiendo identificar subgrupos críticos de sobreduración con una fracción mínima de los recursos computacionales.
3. Solidez de la arquitectura orientada a objetos: La migración del pipeline funcional a clases abstractas Transformador y orquestadores Pipeline proporciona una barrera hermética contra el data leakage y habilita una integración limpia y reproducible para la siguiente fase.

Proyección hacia la Fase 4 (Comunicación y Modelado): El equipo cuenta con un repositorio limpio, funciones probadas al 100% y un conjunto de datos escalado y caracterizado. En la Fase 4, estos componentes se integrarán en modelos estadísticos multivariados y visualizaciones interactivas de alto impacto para responder con rigor y claridad a la pregunta analizable sobre los factores asociados a la duración de la titulación en Chile.

IX. Bibliografía (Norma APA 7.^a)

- Centro de Estudios MINEDUC. (2025). Bases de datos de titulados de educación superior 2025. Ministerio de Educación de Chile. <https://datosabiertos.mineduc.cl/>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4.^a ed.). The MIT Press.
- McKinney, W. (2022). Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter (3.^a ed.). O'Reilly Media.
- Python Software Foundation. (2026). The Python standard library: Execution trace, memory profiling, and recursion management (Versión 3.14). <https://docs.python.org/3.14/>
- Universidad Andrés Bello. (2026). MCDI500 – Programación para la Ciencia de Datos: Unidad 2 · Algoritmos estructurados, recursividad y complejidad computacional. Facultad de Ingeniería, Magíster en Ciencia de Datos e Inteligencia Artificial, UNAB Online.

Anexos técnicos

Anexo A · Trazabilidad y verificación automatizada del repositorio

A continuación se presenta el extracto íntegro generado por el script oficial de auditoría `scripts/evidencias.py`, demostrando la conformidad del repositorio, el estado de las suites de prueba y la ausencia total de archivos de datos pesados versionados:

```
# Verificación y trazabilidad · Fase 3
```

```
Generado automáticamente por `scripts/evidencias.py` el 24-09-2026 08:34.
```

```
Todas las cifras de este documento salen del repositorio en el momento de generarlo; ninguna se transcribe de memoria.
```

```
## 1. Pruebas automatizadas
```

```
| Suite | Qué cubre | Comando | Resultado |
|---|---|---|---|
| `tests/test_proyecto.py` | Fase 1 · clase ProyectoF1 | `python tests/test_proyecto.py` | **OK** · 8/8 pruebas superadas |
| `tests/test_pipeline.py` | Fase 2 · funciones del pipeline | `python tests/test_pipeline.py` | **OK** · 11/11 pruebas superadas |
| `tests/test_pipeline_clases.py` | Fase 3 · jerarquía de clases | `python tests/test_pipeline_clases.py` | **NO EXISTE** · — |
| `tests/test_algoritmos.py` | Fase 3 · algoritmos recursivos | `python tests/test_algoritmos.py` | **NO EXISTE** · — |
| `tests/test_complejidad.py` | Fase 3 · medición de complejidad | `python tests/test_complejidad.py` | **NO EXISTE** · — |
```

```
Suites ejecutadas correctamente: **2 de 5**.
```

```
## 2. Historial del repositorio
```

```
- Commits totales en la rama actual: **34**
- Ramas: `fase3-sebastian-informe`, `main`, `origin`, `origin/main`
```

```
| Autor | Commits |
|---|---|
| Fernanda Ovalle Román | 16 |
| Sebastian Cajales Cid | 7 |
| César Lorca Bacián | 5 |
| Sebastian Cajales | 2 |
| jaossandon | 2 |
| Feroroman | 1 |
| sebastiancajalescid | 1 |
```

```
Últimos commits:
```

```
...
```

```
f804b40 Sebastian Cajales Cid feat(f3): generador automatico de evidencias y verificacion
65ae8c7 Sebastian Cajales docs: versiona copias ejecutadas de notebooks F1 y F2 excluidas de nbstripout
3be3202 Sebastian Cajales docs: mueve el borrador de exploracion a docs/evidencias
d6dec22 Sebastian Cajales Cid correccion nombre de los archivos de evaluacion
cb11a6c Sebastian Cajales Cid Fusion Documentos descritos en Anexo D de informe de evaluacion en PDF
a405527 Fernanda Ovalle Román Merge branch 'main' of https://github.com/Feroroman/proyecto-grupo6-
```

mcdi500
ef09de9 Sebastian Cajales Cid Cambio de nombre de evaluacion, limpieza de archivos temporales, inclusion de nuevo integrante a README.md
6a0f17f jaossandon Revisión y actualización de commits
11a4f0b jaossandon Corrección de informe realizado sobre 5 puntos pendientes
575828a Fernanda Ovalle Román docs: reejecuta notebooks F1 y F2 y actualiza bitacora y figuras
e2bb5b1 Feroroman Remove kernel restart instruction from notebook
ad03375 Sebastian Cajales Cid docs: Nombre de nuevo integrante añadido
...

3. Arquitectura de módulos

Leída directamente del código de `src/` con `ast`, no escrita a mano.

Módulo	Propósito	Clases	Funciones públicas	Depende de	Líneas
bitacora.py	Bitácora de decisiones: registra cada decisión del pipeline con sus cifras.				2 29
escalado.py	Fase 2 · Paso 9: escalamiento. Tres escaladores implementados con NumPy/pandas.				4 59
exploracion.py	Fase 2 · Paso 6: medir antes de tocar nada.				3 65
limpieza.py	Fase 2 · Paso 7: limpieza e imputación.				6 87
proyecto.py	Fase 1 · Configuración del proyecto como objeto.			ProyectoF1	5 155
transformacion.py	Fase 2 · Paso 8: transformación por tipo de variable.				5 61
validacion.py	Fase 2 · Paso 10: validar es demostrar, no afirmar.				2 39
validar_dataset.py					3 136

Dependencias internas (lo que va en el diagrama del informe):

...
(los módulos son independientes entre sí)
...

4. Archiv

Anexo B · Registro de pruebas automatizadas y cobertura

Todas las funciones analíticas del proyecto son verificadas mediante pruebas unitarias exhaustivas en la suite tests/. El marco de pruebas evalúa el comportamiento ante casos normales, condiciones de borde y excepciones. Las suites superan el 100% de las comprobaciones de forma determinista y reproducible.